

Programação Para Dispositivos Móveis II

EXECUÇÃO EM BACKGROUND

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

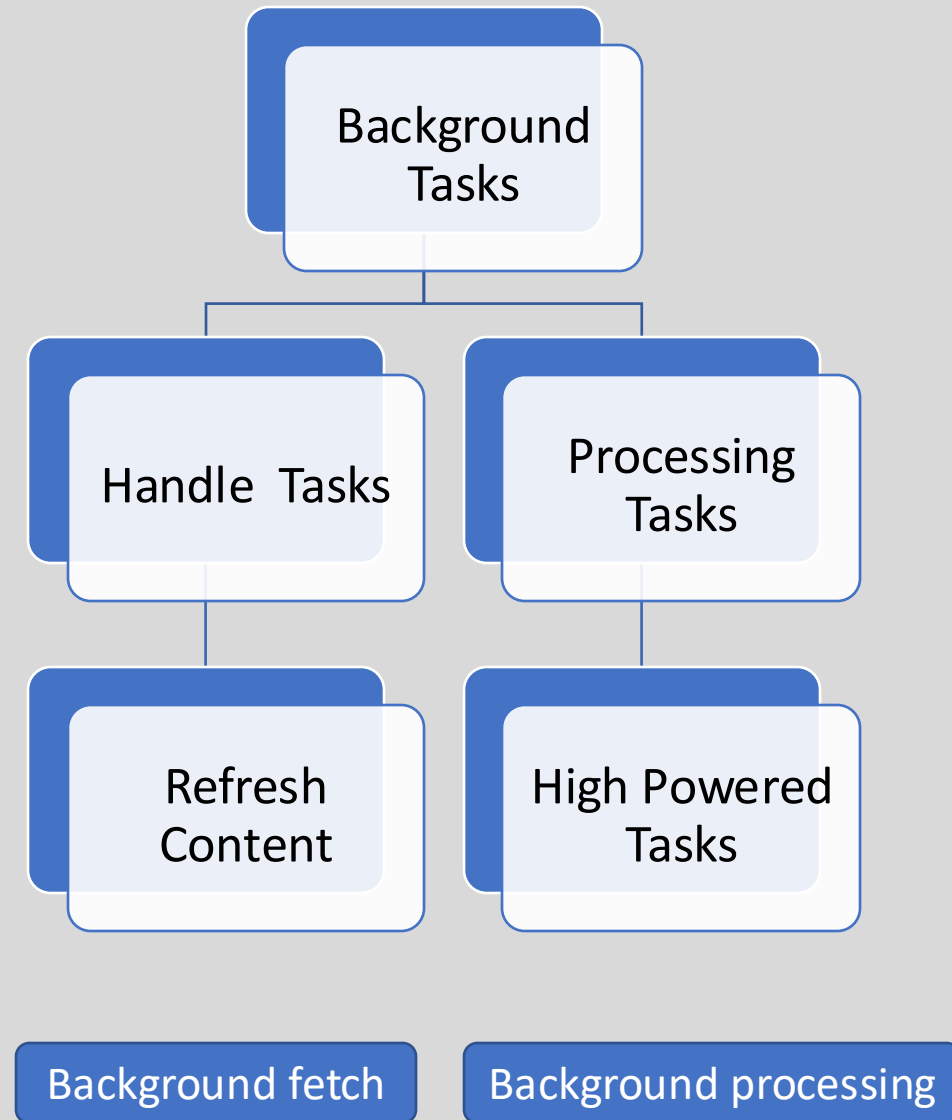
Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jrmr@estg.ipp.pt , Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt

Background Tasks framework

- Permite arrancar aplicações em background
- Manter conteúdo atualizado
- Efetuar tarefas mais demoradas em background
 - Podem requerer (opcionalmente):
 - Ligação à rede
 - Dispositivo em carga



Roadmap

Capabilities

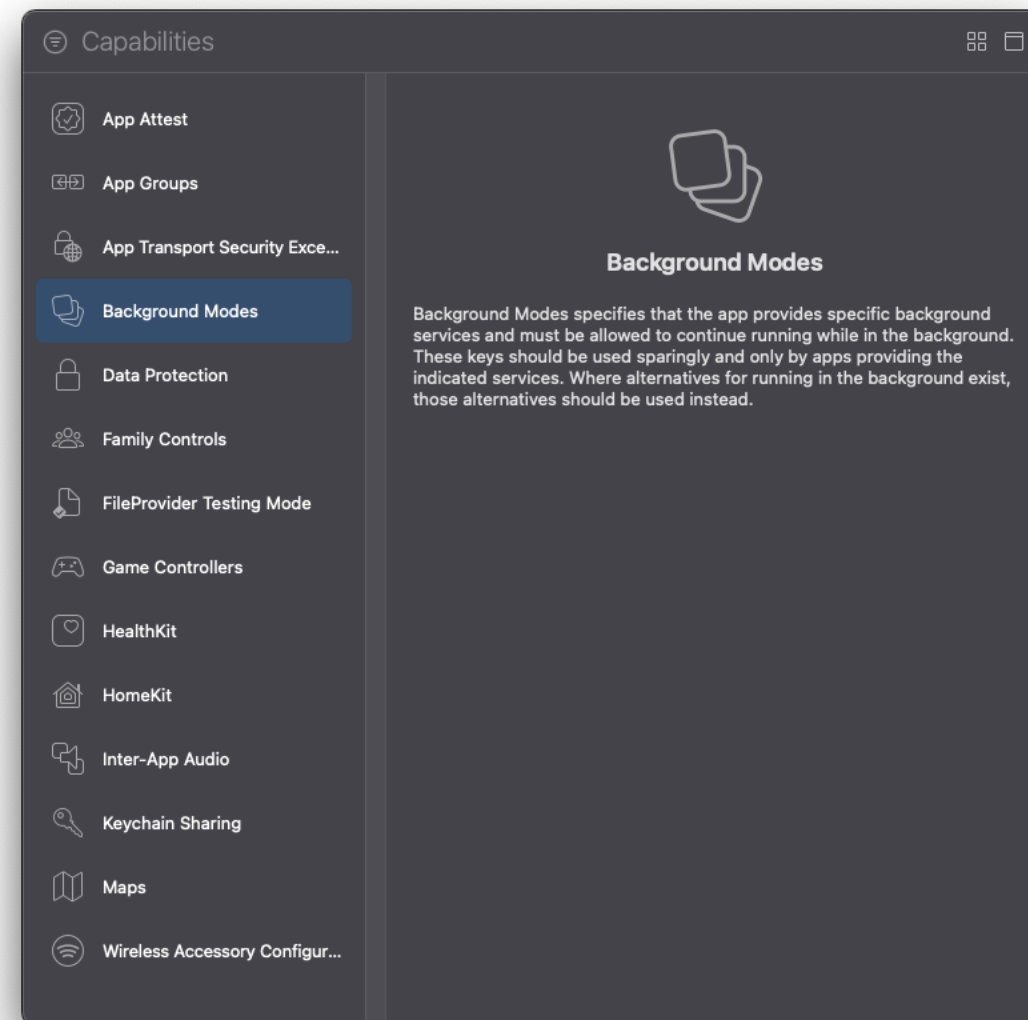


Tasks



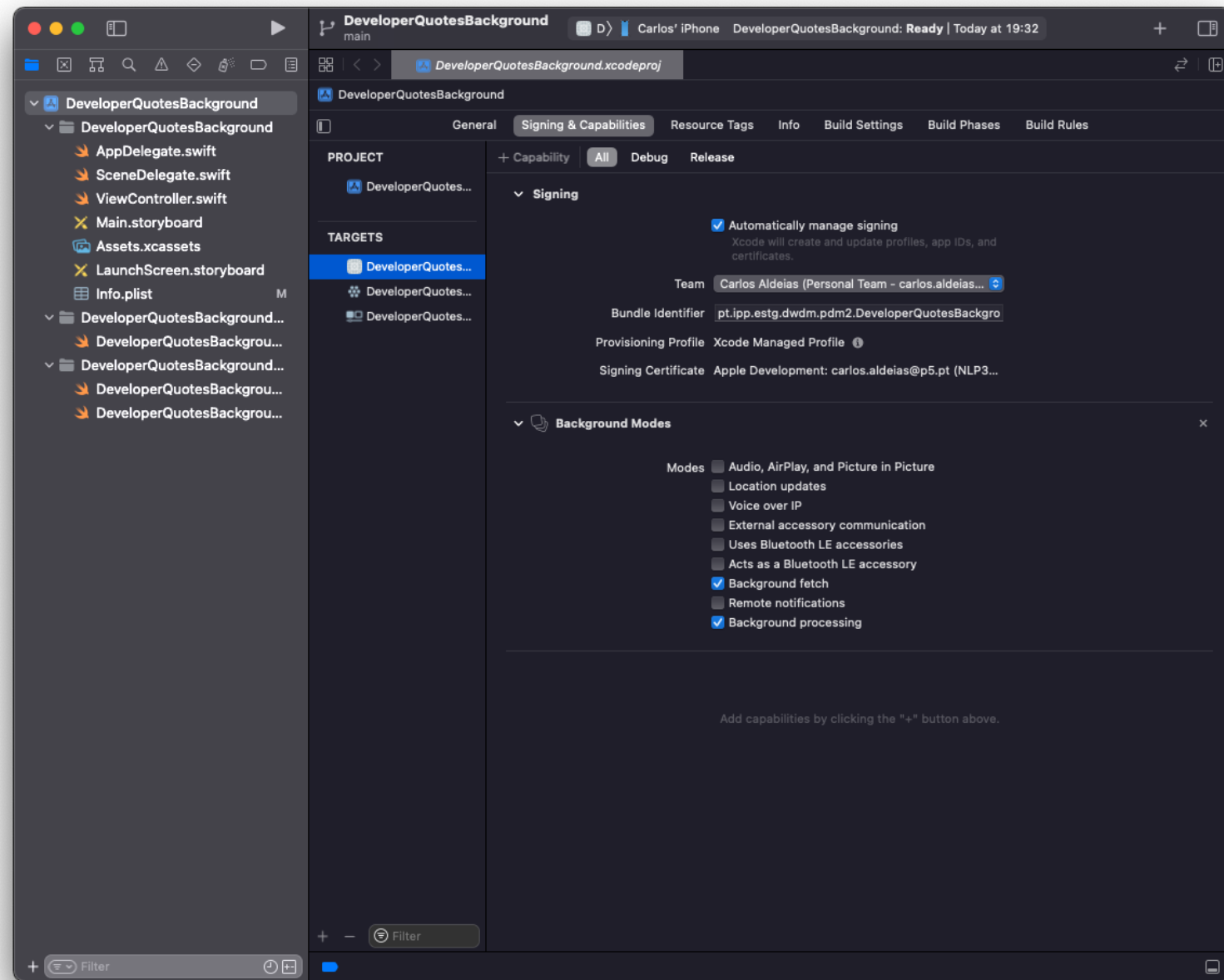
Capabilities

- Declaração de permissões
 - Apple
 - Clientes/utilizadores da aplicação
- Configuração de serviços Apple
 - Push Notifications
 - Apple Pay
 - ...
- As *capabilities* para uma App dependem:
 - Da plataforma
 - Ser membro do Apple Developer Program
- Consultar [Supported capabilities \(iOS\)](#)



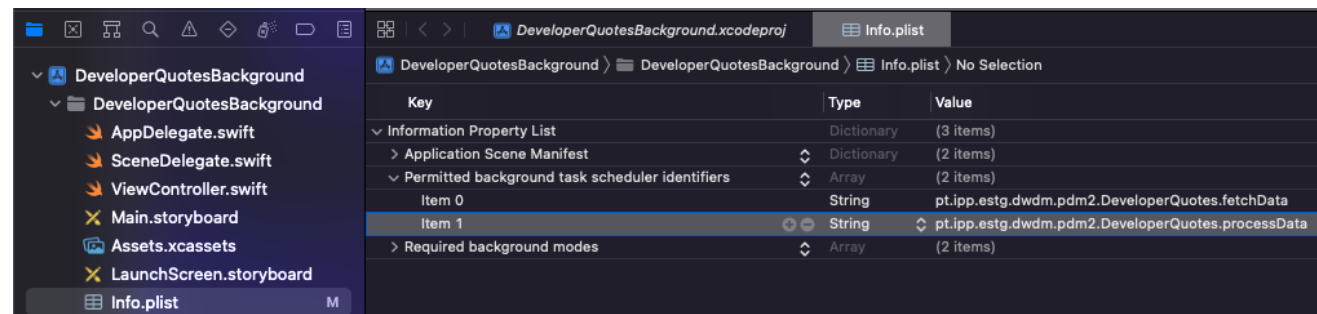
Adding capabilities

- No Project editor:
 - Signing & Capabilities
 - Toolbar, Botão +
 - Background Modes
 - Background fetch
 - Background processing



Registrar tarefas

- Controlar quais as tarefas que serão executadas em background
- No ficheiro Info.plist
 - Adicionar nova linha:
 - "Permitted background task scheduler identifiers" ou [BGTaskSchedulerPermittedIdentifiers](#) array
 - Adicionar os identificadores das tarefas a executar
 - Usar reverse DNS notation
 - Para não chocar com identificadores de outras aplicações
 - Exemplo: pt.ipp.estg.myApp.fetchData



Roadmap

Capabilities



Tasks



Registrar tarefas – AppDelegate.swift

- No ficheiro *AppDelegate.swift*
 - Importar a framework

```
import BackgroundTasks
```

- Declarar uma lista com as operações a serem efetuadas
 - Forma de separar responsabilidades
 - Mantendo informação do progresso

```
let operationQueue = OperationQueue()
```


Registrar tarefas – no arranque da App

- No ficheiro *AppDelegate.swift*, alterar o método que é chamado após a App terminar o arranque da aplicação (*didFinishLaunchingWithOptions*)
- Registrar os handlers que serão chamados quando o sistema executar a tarefa associada ao identifier passado:

```
func application(_ application: UIApplication, didFinishLaunchingWithOptions
[UIApplication.LaunchOptionsKey: Any]?) -> Bool {
    // Override point for customization after application launch.

    // add a register entry per task
    BGTaskScheduler.shared.register(
        forTaskWithIdentifier: "pt.ipp.estg.dwdm.pdm2.DeveloperQuotes.fetchData",
        using: nil) { task in
        self.handleAppRefresh(task: task as! BGAppRefreshTask)
    }

    return true
}
```

launchOptions:

Agendar execução de tarefas

- No ficheiro *AppDelegate.swift*
 - Criar o método que efetua a agendamento de uma tarefa

```
func scheduleAppRefresh() {  
    let request = BGAppRefreshTaskRequest(  
        identifier: "pt.ipp.estg.dwdm.pdm2.DeveloperQuotes.fetchData")  
  
    // Fetch no earlier than 15 minutes from now.  
    request.earliestBeginDate = Date(timeIntervalSinceNow: 15 * 60)  
  
    do {  
        try BGTaskScheduler.shared.submit(request)  
    } catch {  
        print("Could not schedule app refresh: \(error)")  
    }  
}
```

Execução de tarefas

- No ficheiro *AppDelegate.swift*
 - Criar o método que efetua a execução de uma tarefa

```
func handleAppRefresh(task: BGAppRefreshTask) {  
    // Schedule a new refresh task.  
    scheduleAppRefresh()  
  
    // Create an operation that performs the main part of the background task.  
    let operation = FetchRandomQuoteOperation()  
  
    // Provide the background task with an expiration handler that cancels the operation.  
    task.expirationHandler = {  
        operation.cancel()  
    }  
  
    // Inform the system that the background task is complete when the operation completes.  
    operation.completionBlock = {  
        task.setTaskCompleted(success: !operation.isCancelled)  
    }  
  
    // Start the operation.  
    operationQueue.addOperation(operation)  
}
```

Operação principal

- Criar um novo ficheiro .swift

Exemplo simples

```
import Foundation

class FetchRandomQuoteOperation: Operation {
    var result: Quote?

    override func main() {
        result = QuoteManager.fetchRandomQuote()
    }
}
```

Using Core Data

```
import Foundation

class FetchRandomQuoteOperation: Operation {
    private let context: NSManagedObjectContext
    var result: Quote?

    init(context: NSManagedObjectContext) {
        self.context = context
    }

    override func main() {
        let request: NSFetchRequest<Quote> = Quote.fetchRequest()
        context.performAndWait {
            do {
                let fetchResult = try context.fetch(request)
                guard !fetchResult.isEmpty else { return }

                result = fetchResult[0]
            } catch {
                print("Error fetching from context: \(error)")
            }
        }
    }
}
```

Next Milestone

- Notificações
 - User Notifications
 - Push Notifications
 - Notification Center



Leitura recomendada

- **UIKit Framework** <https://developer.apple.com/documentation/uikit>
- **Background Tasks** <https://developer.apple.com/documentation/backgroundtasks>
- **Supported capabilities (iOS)** <https://developer.apple.com/help/account/reference/supported-capabilities-ios/>

Programação Para Dispositivos Móveis II

EXECUÇÃO EM BACKGROUND

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jrmr@estg.ipp.pt , Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt